10. Misc.

Contents

Tree Knapsack

```cpp
vector<int> merge(vector<int> &a, vector<int> &b) {
    vector<int> res;
    res.resize(a.size() + b.size() - 1, INF);
    for (int i = 0; i < a.size(); i++) {
        if (a[i] == INF) continue;
        for (int j = 0; j < b.size(); j++) {
            res[i + j] = min(res[i + j], a[i] + b[j]);
        }
    }
    return res;
}
```

FFT / NTT

```cpp
namespace FFT {
    // [1. CONSTANTS] Modulo and its primitive roots for NTT
    const int mod = 998244353; // 998244353 754974721 167772161
    const int root = 3; // 3 11 3
    const int invRoot = 332748118; // 332748118 617706590 55924054

    inline int mul(int x, int y) { return int(x * 1LL * y % mod); }
    inline int add(int x, int y) { return x + y < mod? x + y: x + y - mod; }
    inline int sub(int x, int y) { return x - y < 0? x - y + mod: x - y; }
    int fp(int b, int e) {
        int res = 1;
        while(e) {
            if(e & 1) res = mul(res, b);
            b = mul(b, b), e >>= 1;
        }
        return res;
    }

    // [3. GENERATOR] O(sqrt(mod) log mod) - Run locally to find
    // root/invRoot for a new prime mod
    void primitiveRoot() {
        int phi = mod - 1;
        vector<int> fac;
        for(int i = 2; i * 1LL * i < phi; i++) {
            if(phi % i == 0) {
                fac.push_back(i);
                while(phi % i == 0) phi /= i;
            }
        }
        if(phi > 1) fac.push_back(phi);

        for(int g = 2; g < mod; g++) {
            for(int pr : fac) if(fp(g, (mod - 1) / pr) == 1)
                goto bad;
            cout << "const␣int␣root␣=␣" << g << ";\n";
            cout << "const␣int␣invRoot␣=␣" << fp(g, mod - 2) << ";\n";
            return;
            bad:;
        }
        cerr << "404\n";
```

```
    }

    using cd = complex<double>;
    double pi = acos(-1);

    // [4. FFT CORE] O(N log N) - Used internally by complex multiplication
    void fft(vector<cd> &a, bool invert) {
        int n = (int)a.size();
        for (int i = 1, j = 0; i < n; i++) {
            j ^= ((1 << __lg(n - 1 ^ j)) - 1) ^ (n - 1);
            if(i < j)swap(a[i], a[j]);
        }
        double ang = pi * (invert ? -1 : 1);
        for (int len = 1; len < n; len <<= 1, ang /= 2) {
            cd w1(cos(ang), sin(ang));
            for (int i = 0; i < n; i += len * 2) {
                cd w(1), u, v;
                for(int j = 0; j < len; j++) {
                    u = a[i + j], v = a[i + j + len] * w, w *= w1;
                    a[i + j] = u + v, a[i + j + len] = u - v;
                }
            }
        }
        if(invert) for(cd &x : a) x /= n;
    }

    // [5. FFT MULTIPLY] O(N log N) - Multiplies polynomials without
    // modulo. Returns exact integers.
    // Packs 'a' into real and 'b' into imag to save one FFT pass.
    vector<int64_t> mul(vector<int> const &a, vector<int> const &b) {
        int N = 1;
        while (N < a.size() + b.size() - 1) N <<= 1;
        vector<cd> t(N);
        for(int i = 0; i < a.size(); i++) t[i].real(a[i]);
        for(int i = 0; i < b.size(); i++) t[i].imag(b[i]);
        fft(t, false);
        for(auto &x : t) x *= x;
        fft(t, true);
        vector<int64_t> ans(N);
        for(int i = 0; i < N; i++) ans[i] = (int64_t)round(t[i].imag() / 2.0);
        return ans;
    }

    // [6. WILDCARD MATCHING] O(N log N) - Returns starting indices of
    // pattern 't' in text 's'.
    // Supports '?' as wildcard. Returns 0-based indices.
    vector<int> string_matching(string &s, string &t) {
        if (t.size() > s.size()) return {};
        int n = s.size(), m = t.size();
        vector<int> s1(n), s2(n), s3(n);
        for(int i = 0; i < n; i++) {
            // assign any non-zero number for non '?'s
            s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1;
            s2[i] = s1[i] * s1[i];
            s3[i] = s1[i] * s2[i];
        }
```

```
        vector<int> t1(m), t2(m), t3(m);
        for(int i = 0; i < m; i++) {
            t1[i] = t[m - i - 1] == '?' ? 0 : t[m - i - 1] - 'a' + 1;
            t2[i] = t1[i] * t1[i];
            t3[i] = t1[i] * t2[i];
        }
        auto s1t3 = mul(s1, t3);
        auto s2t2 = mul(s2, t2);
        auto s3t1 = mul(s3, t1);
        vector<int> oc;
        for(int i = m - 1; i < n; i++)
            if(s1t3[i] - s2t2[i] * 2 + s3t1[i] == 0)
                oc.push_back(i - m + 1);
        return oc;
    }

    // [7. NTT CORE] O(N log N) - Used internally by modular multiplication
    void ntt(vector<int> &a, bool invert) {
        int n = (int)a.size();
        for (int i = 1, j = 0; i < n; i++) {
            j ^= ((1 << __lg(n - 1 ^ j)) - 1) ^ (n - 1);
            if(i < j)swap(a[i], a[j]);
        }
        for(int len = 2, l2 = 1, w1, w, u, v; len <= n; len <<= 1, l2 <<= 1) {
            w1 = fp(invert? invRoot: root, (mod - 1) / len);
            for(int i = 0; i < n; i += len) {
                w = 1;
                for(int j = 0; j < l2; j++) {
                    u = a[i + j], v = mul(a[i + j + l2], w), w = mul(w, w1);
                    a[i + j] = add(u, v), a[i + j + l2] = sub(u, v);
                }
            }
        }
        if (invert) {
            int n_1 = fp(n, mod - 2);
            for(int & x : a) x = mul(x, n_1);
        }
    }

    // [8. NTT MULTIPLY] O(N log N) - Multiplies polynomials modulo 'mod'.
    vector<int> mulMod(vector<int> a, vector<int> b) {
        int N = 1, sz = a.size() + b.size();
        while (N < a.size() + b.size() - 1) N <<= 1;
        a.resize(N);
        b.resize(N);
        ntt(a, false), ntt(b, false);
        for(int i = 0; i < N; i++)
            a[i] = int(a[i] * 1LL * b[i] % mod);
        ntt(a, true);
        a.resize(sz - 1);
        return a;
    }

    // [9. FWHT AND/OR/XOR] O(N log N) - Bitwise convolutions.
    // N must be power of 2.
    void fwht_and(vector<ll>& a, bool invert) {
```

```
        int n = a.size();
        for (int len = 1; 2 * len <= n; len <<= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; ++j) {
                    a[i + j] = (a[i + j] +
                        (invert? -1: 1) * a[i + j + len] + mod) % mod;
                }
            }
        }
    }
    void fwht_or(vector<ll>& a, bool invert) {
        int n = a.size();
        for (int len = 1; 2 * len <= n; len <<= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; ++j) {
                    a[i + j + len] = (a[i + j + len] +
                        (invert? -1: 1) * a[i + j] + mod) % mod;
                }
            }
        }
    }
    void fwht_xor(vector<ll>& a, bool invert) {
        int n = a.size();
        for (int len = 1; 2 * len <= n; len <<= 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; ++j) {
                    ll u = a[i + j], v = a[i + j + len];
                    a[i + j] = (u + v) % mod;
                    a[i + j + len] = (u - v + mod) % mod;
                }
            }
        }
        if (invert) {
            ll inv2 = (mod + 1) / 2;
            ll inv_n = 1;
            for(int i = 1; i < n; i <<= 1)
                inv_n = inv_n * inv2 % mod;
            for (ll &x : a) x = x * inv_n % mod;
        }
    }

    // [10. CONVOLUTION RUNNER] O(N log N) - Pass fwht_and,
    // fwht_or, or fwht_xor as 'fun'.
    // Solves for C[k] = sum(A[i] * B[j]) where i (op) j = k.
    template<typename F>
    vector<ll> convolution(vector<ll> a, vector<ll> b, F const &fun) {
        int n = 1;
        while (n < max(a.size(), b.size())) n <<= 1;
        a.resize(n), b.resize(n); // Resizes automatically to next power of 2
        fun(a, false);
        fun(b, false);
        for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
        fun(a, true);
        return a;
    }
}
```

Segmented Seive

```
vector<int> simple_sieve(int n) {
    vector<bool> is_prime(n + 1, true);
    vector<int> primes;
    if (n >= 0) is_prime[0] = false;
    if (n >= 1) is_prime[1] = false;
    for (int i = 2; i <= n; i++) {
        if (!is_prime[i]) continue;
        primes.push_back(i);
        if (1LL * i * i <= n) {
            for (ll j = 1LL * i * i; j <= n; j += i) {
                is_prime[j] = false;
            }
        }
    }
    return primes;
}

vector<bool> segmented_sieve_mask(ll l, ll r) {
    if (l > r) return {};
    int root = sqrtl(r);
    while (1LL * (root + 1) * (root + 1) <= r) root++;
    while (1LL * root * root > r) root--;
    vector<int> primes = simple_sieve(root);
    vector<bool> is_prime(r - l + 1, true);
    for (ll p: primes) {
        ll start = max(p * p, ((l + p - 1) / p) * p);
        for (ll j = start; j <= r; j += p) {
            is_prime[j - l] = false;
        }
    }
    for (ll x = l; x <= min(r, 1LL); x++) {
        is_prime[x - l] = false;
    }
    return is_prime;
}

// returns all primes in [l, r]
vector<ll> segmented_sieve(ll l, ll r) {
    vector<bool> is_prime = segmented_sieve_mask(l, r);
    vector<ll> primes;
    for (ll i = 0; i < (ll) is_prime.size(); i++) {
        if (is_prime[i]) primes.push_back(l + i);
    }
    return primes;
}
```

Hilbert Order

```
vector<pair<int64_t, Query> > blc; // {hilbert_order_id, Query}
// pow = __lg(n) + 1, rot = 0
function<int64_t(int, int, int, int)> hilbert_order =
    [&](int x, int y, int pow, int rot) -> int64_t {
    if (pow == 0) return 0;
```

```
    int h_pow = 1 << (pow - 1);
    int seg = ((x < h_pow ? (y < h_pow ? 0 : 3) : (y < h_pow ? 1 : 2)) + rot) & 3;
    const static int rotate_delta[] = {3, 0, 0, 1};
    int nx = x & (x ^ h_pow), ny = y & (y ^ h_pow), n_rot = (rot +
        rotate_delta[seg]) & 3;
    int64_t sub_square_size = 1LL << (2 * pow - 2), add =
        hilbert_order(nx, ny, pow - 1, n_rot);
    return seg * 1LL * sub_square_size + (seg == 1 || seg == 2 ? add :
        sub_square_size - add - 1);
};
hilbert_order(l, r, __lg(n) + 1, 0);
```

Wavelet Matrix

```
/*
 * Wavelet Matrix
 *
 * Supports:
 *   - count_less(l, r, x): count numbers < x in a[l..r]
 *   - sum_less(l, r, x): sum of numbers < x in a[l..r]
 *   - range_count(l, r, x, y): count numbers in [x, y]
 *   - range_sum(l, r, x, y): sum of numbers in [x, y]
 *
 * Build: O(n log MaxValue)
 * Query: O(log MaxValue)
 * Memory: O(n log MaxValue)
 */

struct WaveletMatrix {
    int n, N, LOG;
    vector<int> bv, mid;
    vector<long long> ps;

    WaveletMatrix(const vector<int>& orig_a) : n(orig_a.size()),
        N(orig_a.size() + 1) {
        if (!n) { LOG = 0; return; }
        int mx = *max_element(orig_a.begin(), orig_a.end());
        LOG = mx > 0 ? __lg(mx) + 1 : 1;

        bv.assign(LOG * N, 0);
        ps.assign((LOG + 1) * N, 0);
        mid.resize(LOG);

        vector<int> a = orig_a, nxt(n);

        for (int lvl = LOG - 1; lvl >= 0; --lvl) {
            int pre = lvl * N, off = (lvl + 1) * N;
            int ones = 0;

            // Merge prefix sum and bit-counting passes
            for (int i = 0; i < n; i++) {
                ps[off + i + 1] = ps[off + i] + a[i];
                int b = (a[i] >> lvl) & 1;
                bv[pre + i + 1] = bv[pre + i] + b;
                ones += b;
```

```
            }

            int p0 = 0, p1 = mid[lvl] = n - ones;
            for (int i = 0; i < n; i++) {
                if ((a[i] >> lvl) & 1) nxt[p1++] = a[i];
                else nxt[p0++] = a[i];
            }
            a = nxt;
        }
        for (int i = 0; i < n; i++)
            ps[i + 1] = ps[i] + a[i];
    }

    int count_less(int L, int R, int x) {
        if (L > R || x <= 0 || !LOG) return 0;
        if (x >> LOG) return R - L + 1;

        R++;
        int ans = 0;

        for (int lvl = LOG - 1; lvl >= 0; --lvl) {
            int pre = lvl * N;
            int oL = bv[pre + L], oR = bv[pre + R];
            int zL = L - oL, zR = R - oR;

            if ((x >> lvl) & 1) {
                ans += zR - zL;
                L = mid[lvl] + oL;
                R = mid[lvl] + oR;
            } else {
                L = zL; R = zR;
            }
        }
        return ans;
    }

    long long sum_less(int L, int R, int x) {
        if (L > R || x <= 0 || !LOG) return 0;
        if (x >> LOG) return ps[LOG * N + R + 1] - ps[LOG * N + L];

        R++;
        long long ans = 0;

        for (int lvl = LOG - 1; lvl >= 0; --lvl) {
            int pre = lvl * N;
            int oL = bv[pre + L], oR = bv[pre + R];
            int zL = L - oL, zR = R - oR;

            if ((x >> lvl) & 1) {
                ans += ps[pre + zR] - ps[pre + zL];
                L = mid[lvl] + oL;
                R = mid[lvl] + oR;
            } else {
                L = zL; R = zR;
            }
        }
        return ans;
    }
```

```

    int range_count(int L, int R, int x, int y) {
        if (x > y) return 0;
        return count_less(L, R, y + 1) - count_less(L, R, x);
    }

    long long range_sum(int L, int R, int x, int y) {
        if (x > y) return 0;
        return sum_less(L, R, y + 1) - sum_less(L, R, x);
    }
};
```

BCC

```
struct BCC {
    int n;
    vector<int> id;
    vector<bool> isArt;
    vector<vector<int> > tree, g;

    explicit BCC(int n) : n(n), g(n) {
    }

    void addEdge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void init() {
        id.assign(n, -1);
        int m = 0, cnt = 0;
        vector<int> in(n, -1), low(n, -1), st;
        st.reserve(n);
        vector<vector<int> > bcc(n);
        function<void(int, int)> dfs = [&](int u, int p) -> void {
            in[u] = low[u] = cnt++, st.push_back(u);
            for (int v: g[u]) {
                if (!~in[v]) {
                    dfs(v, u), low[u] = min(low[u], low[v]);
                    if (low[v] >= in[u]) {
                        int x = -1;
                        while (x ^ v) x = st.back(), st.pop_back(),
                            bcc[x].push_back(m);
                        bcc[u].push_back(m++);
                    }
                } else if (v != p) low[u] = min(low[u], in[v]);
            }
        };
        dfs(0, -1);
        for (int u = 0; u < n; u++) {
            if (bcc[u].size() == 1) id[u] = bcc[u].front();
            else id[u] = m++;
        }
        tree.assign(m, vector<int>()), isArt.assign(m, false);
        for (int u = 0; u < n; u++) {
```

```
            if (bcc[u].size() ^ 1) {
                isArt[id[u]] = true;
                for (int v: bcc[u]) {
                    tree[id[u]].push_back(v);
                    tree[v].push_back(id[u]);
                }
            }
        }
    }
};
```

Blossom

```
/*
 * Blossom / Edmonds' algorithm for maximum matching in a general undirected graph.
 *
 * Usage:
 *   - Set n and g.
 *   - g must be an undirected adjacency list.
 *   - Call findMaximumMatching().
 *   - Result is stored in match[]:
 *       match[v] = paired vertex, or -1 if unmatched.
 *
 * Complexity:
 *   - O(n^3) in the common competitive programming implementation below.
 *   - Suitable for general graphs with up to a few hundred vertices.
 *   - Needed only for non-bipartite matching; for bipartite graphs, use Hopcroft-Karp.
 */


int n;
vector<vector<int>> g;
vector<int> match, p, base;
vector<bool> used, blossom;

int lca(int a, int b) {
    vector<bool> vis(n, false);
    for(;;) {
        a = base[a], vis[a] = true;
        if(match[a] == -1) break;
        a = p[match[a]];
    }
    for(;;) {
        b = base[b];
        if(vis[b]) return b;
        b = p[match[b]];
    }
}

void markPath(int v, int b, int x) {
    while (base[v] != b) {
        blossom[base[v]] = blossom[base[match[v]]] = true;
        p[v] = x;
        x = match[v];
        v = p[match[v]];
    }
```

```
}

int findPath(int src) {
    used.assign(n, false);
    p.assign(n, -1);
    iota(base.begin(), base.end(), 0);
    queue<int> q;
    q.push(src);
    used[src] = true;

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int u: g[v]) {
            if (base[v] == base[u] || match[v] == u) continue;
            if (u == src || (match[u] != -1 && p[match[u]] != -1)) {
                int cur = lca(v, u);
                blossom.assign(n, false);
                markPath(v, cur, u);
                markPath(u, cur, v);
                for (int i = 0; i < n; i++) {
                    if (blossom[base[i]]) {
                        base[i] = cur;
                        if (!used[i]) used[i] = true, q.push(i);
                    }
                }
            } else if (p[u] == -1) {
                p[u] = v;
                if (match[u] == -1) return u;
                used[match[u]] = true;
                q.push(match[u]);
            }
        }
    }
    return -1;
}

void findMaximumMatching() {
    match.assign(n, -1);
    base.resize(n);
    for (int i = 0; i < n; i++) {
        if (match[i] == -1) {
            int v = findPath(i);
            while (v != -1) {
                int pv = p[v], w = match[pv];
                match[v] = pv, match[pv] = v, v = w;
            }
        }
    }
}
void recover(vector<pair<int, int>>& ans) {
    vector<short> vis(n);
    for (int i = 0; i < n; i++) {
        if (!vis[i] && ~match[i]) {
            vis[i] = vis[match[i]] = 1;
            ans.emplace_back(i, match[i]);
```

```
        }
    }
}
```

Dynamic Bitset

```
#include <tr2/dynamic_bitset>
tr2::dynamic_bitset<long long> bit(N);
```

Dynamic Suffix Array

```
#pragma GCC optimize("O3,unroll-loops")

uint64_t fast_rand() {
    static uint64_t x = 88172645463325252ULL;
    x ^= x << 13; x ^= x >> 7; x ^= x << 17;
    return x;
}

class DynamicSuffixArray {
    struct Node {
        int l = 0, r = 0, p = 0, sz = 1;
        uint64_t y = fast_rand();
    };

    vector<char> s;
    vector<Node> tr;
    int root_ = 0;

    void upd(int x) { if (x) tr[x].sz = 1 + tr[tr[x].l].sz + tr[tr[x].r].sz; }

    int setP(int x, int p) {
        if (x) tr[x].p = p;
        return x;
    }

    int merge(int L, int R) {
        if (!L || !R) return L + R;
        if (tr[L].y < tr[R].y) return setP(tr[R].l = merge(L, tr[R].l), R), upd(R), R;
        setP(tr[L].r = merge(tr[L].r, R), L);
        return upd(L), L;
    }

    int getIndex(int x) {
        if (!x) return 0;
        int i = tr[tr[x].l].sz + 1;
        while (tr[x].p) {
            int p = tr[x].p;
            if (x == tr[p].r) i += tr[tr[p].l].sz + 1;
            x = p;
        }
        return i;
    }

```

```
    int compare(int id1, int id2, int i) {
        int cmp = s[id1] - s[id2];
        return cmp == 0 ? i - getIndex(id2 - 1) : cmp;
    }

    void split(int x, int id, int i, int& L, int& R) {
        if (!x) { L = R = 0; return; }
        if (compare(id, x, i) < 0) {
            split(tr[x].l, id, i, L, tr[x].l);
            setP(tr[x].l, x);
            R = x;
        } else {
            split(tr[x].r, id, i, tr[x].r, R);
            setP(tr[x].r, x);
            L = x;
        }
        upd(x);
    }

    int insert(int x, int id, int i1) {
        if (!x) return id;
        if (tr[x].y < tr[id].y) {
            split(x, id, i1, tr[id].l, tr[id].r);
            setP(tr[id].l, id);
            setP(tr[id].r, id);
            return upd(id), id;
        }
        if (compare(id, x, i1) < 0) setP(tr[x].l = insert(tr[x].l, id, i1), x);
        else setP(tr[x].r = insert(tr[x].r, id, i1), x);
        tr[x].sz++;
        return x;
    }

public:
    DynamicSuffixArray(int max_capacity = 100000) {
        s.reserve(max_capacity + 10);
        tr.reserve(max_capacity + 10);
        s.push_back(0);
        tr.emplace_back();
        tr[0].sz = 0;
    }

    void push_front(char c) {
        s.push_back(c);
        tr.emplace_back();
        int id = size();
        root_ = insert(root_, id, getIndex(id - 1));
    }

    char pop_front() {
        if (size() <= 0) return 0;
        int id = size(), p = tr[id].p, tmp = merge(tr[id].l, tr[id].r);
        setP(tmp, p);
        if (p) {
            (tr[p].l == id ? tr[p].l : tr[p].r) = tmp;
            while (p) tr[p].sz--, p = tr[p].p;
```

```
        } else root_ = tmp;

        tr.pop_back();
        char c = s.back();
        s.pop_back();
        return c;
    }
    // count number of occurences
    int match(const string& q, int x = -1, bool L = false, bool R = false) {
        if (x == -1) x = root_;
        if (!x) return 0;
        if (L && R) return tr[x].sz;
        for (size_t i = 0; i < q.size(); ++i) {
            if (q[i] > s[x - i]) return match(q, tr[x].r, false, R);
            if (q[i] < s[x - i]) return match(q, tr[x].l, L, false);
        }
        return 1 + match(q, tr[x].l, L, true) + match(q, tr[x].r, true, R);
    }

    int size() { return (int)s.size() - 1; }
    char at(int i) { return s[size() - i]; }
};
```

Dynamic Suffix Array With LCS

```
static uint64_t rsd = chrono::steady_clock::now().time_since_epoch().count();
inline uint64_t fast_rand() {
    rsd ^= rsd << 13; rsd ^= rsd >> 7; rsd ^= rsd << 17;
    return rsd;
}

class DynamicSuffixArray {
    using u64 = uint64_t; using u128 = __int128_t;

    // Secure prime modulo to prevent anti-hash WA
    static constexpr u64 MOD = (1ULL << 61) - 1;
    static constexpr u64 BASE = 11995408973635179863ULL % MOD;

    static inline u64 add(u64 a, u64 b) { return a + b >= MOD ? a + b - MOD : a + b; }
    static inline u64 sub(u64 a, u64 b) { return a >= b ? a - b : a + MOD - b; }
    static inline u64 mul(u64 a, u64 b) {
        u128 r = (u128)a * b;
        u64 res = (u64)(r >> 61) + (u64)(r & MOD);
        return res >= MOD ? res - MOD : res;
    }

    struct Node {
        int l = 0, r = 0, p = 0, sz = 1;
        int mn = 0, mx = 0;
        u64 y; Node() : y(fast_rand()) {}
    };
    vector<char> s; vector<Node> tr; vector<u64> hp, pw;
    int root_ = 0;

    void ensure_powers(int n) {
        while ((int)pw.size() <= n)
```

```
        pw.push_back(mul(pw.back(), BASE));
    }

    inline void upd(int x) {
        if (!x) return;
        tr[x].sz = 1 + tr[tr[x].l].sz + tr[tr[x].r].sz;
        tr[x].mn = tr[x].mx = x;
        if (tr[x].l) {
            tr[x].mn = min(tr[x].mn, tr[tr[x].l].mn);
            tr[x].mx = max(tr[x].mx, tr[tr[x].l].mx);
        }
        if (tr[x].r) {
            tr[x].mn = min(tr[x].mn, tr[tr[x].r].mn);
            tr[x].mx = max(tr[x].mx, tr[tr[x].r].mx);
        }
    }

    inline int setP(int x, int p) {
        if (x) tr[x].p = p;  return x;
    }

    int merge(int L, int R) {
        if (!L || !R) return L + R;
        if (tr[L].y < tr[R].y)
            return setP(tr[R].l = merge(L, tr[R].l), R), upd(R), R;
        return setP(tr[L].r = merge(tr[L].r, R), L), upd(L), L;
    }

    int getIndex(int x) {
        if (!x) return 0;
        int i = tr[tr[x].l].sz + 1;
        while (tr[x].p) {
            int p = tr[x].p;
            if (x == tr[p].r) i += tr[tr[p].l].sz + 1;
            x = p;
        }
        return i;
    }

    int compare(int id1, int id2, int i) {
        int cmp = s[id1] - s[id2];
        return cmp == 0 ? i - getIndex(id2 - 1) : cmp;
    }

    void split(int x, int id, int i, int &L, int &R) {
        if (!x) { L = R = 0; return; }
        if (compare(id, x, i) < 0) {
            split(tr[x].l, id, i, L, tr[x].l);
            setP(tr[x].l, x);
            R = x;
        } else {
            split(tr[x].r, id, i, tr[x].r, R);
            setP(tr[x].r, x);
            L = x;
        } upd(x);
    }
```

```
88
89      int insert(int x, int id, int i1) {
90          if (!x) return id;
91          if (tr[x].y < tr[id].y) {
92              split(x, id, i1, tr[id].l, tr[id].r);
93              setP(tr[id].l, id);
94              setP(tr[id].r, id);
95              return upd(id), id;
96          }
97          if (compare(id, x, i1) < 0) setP(tr[x].l = insert(tr[x].l, id, i1), x);
98          else setP(tr[x].r = insert(tr[x].r, id, i1), x);
99          tr[x].sz++; return x;
100     }
101
102     inline u64 str_hash(const vector<u64> &h, int l, int len) const {
103         return sub(h[l + len], mul(h[l], pw[len]));
104     }
105
106     vector<u64> make_hash(const string &q) const {
107         vector<u64> h(q.size() + 1);
108         for (size_t i = 0; i < q.size(); ++i)
109             h[i + 1] = add(mul(h[i], BASE), (unsigned char)q[i]);
110         return h;
111     }
112
113     int lcp_query_node(const string &q, const vector<u64> &qh, int qi, int x) const {
114         int lo = 0, hi = min((int)q.size() - qi, x);
115         while (lo < hi) {
116             int mid = (lo + hi + 1) >> 1;
117             u64 h_q = str_hash(qh, qi, mid);
118             u64 h_s = sub(hp[x], hp[x - mid]);
119             if (mul(h_q, pw[x - mid]) == h_s) lo = mid;
120             else hi = mid - 1;
121         }
122         return lo;
123     }
124
125     inline int lcp_query_node_fast(const string &q, const vector<u64> &qh,
126         int qi, int x) const {
127             return !x ? 0 : lcp_query_node(q, qh, qi, x);
128     }
129
130     int compare_prefix(const string &q, const vector<u64> &qh, int qi, int x) const {
131         int l = lcp_query_node(q, qh, qi, x);
132         if (l == (int)q.size() - qi) return 0; // q is prefix of x
133         if (l == x) return 1;                  // x is prefix of q => q > x
134         return q[qi + l] < s[x - l] ? -1 : 1;
135     }
136
137     int compare_query_node(const string &q, const vector<u64> &qh, int qi, int x)
138     const {
139         int l = lcp_query_node(q, qh, qi, x);
140         int qlen = (int)q.size() - qi, slen = x;
141         if (l == qlen && l == slen) return 0;
142         if (l == qlen) return -1;
143         if (l == slen) return 1;
```

```
        return q[qi + l] < s[x - l] ? -1 : 1;
    }

    void split_lower(int x, const string &q, const vector<u64> &qh, int qi,
        int &L, int &R) {
        if (!x) { L = R = 0; return; }
        if (compare_prefix(q, qh, qi, x) > 0) { // q > suffix -> suffix belongs in L
            split_lower(tr[x].r, q, qh, qi, tr[x].r, R);
            setP(tr[x].r, x); L = x;
        } else {
            split_lower(tr[x].l, q, qh, qi, L, tr[x].l);
            setP(tr[x].l, x); R = x;
        }
        upd(x);
    }

    void split_upper(int x, const string &q, const vector<u64> &qh, int qi, int &L,
        int &R) {
        if (!x) { L = R = 0; return; }
        // q >= suffix (includes prefix match) -> belongs in L
        if (compare_prefix(q, qh, qi, x) >= 0) {
            split_upper(tr[x].r, q, qh, qi, tr[x].r, R);
            setP(tr[x].r, x); L = x;
        } else {
            split_upper(tr[x].l, q, qh, qi, L, tr[x].l);
            setP(tr[x].l, x); R = x;
        }
        upd(x);
    }

    struct MatchResult { int cnt, first_occ, last_occ; };

    MatchResult get_matches(const string &q) {
        if (q.empty() || size() == 0) return {0, -1, -1};
        ensure_powers(max(size(), (int)q.size()));
        int L = 0, M = 0, R = 0, tmp = 0;
        vector<u64> qh = make_hash(q);
        split_lower(root_, q, qh, 0, L, tmp);
        split_upper(tmp, q, qh, 0, M, R);
        MatchResult res = {0, -1, -1};
        if (M) {
            res.cnt = tr[M].sz;
            res.first_occ = size() - tr[M].mx;
            res.last_occ = size() - tr[M].mn;
        }
        root_ = merge(L, merge(M, R));
        return res;
    }

    pair<int, int> pred_succ(const string &q, const vector<u64> &qh, int qi) const {
        int x = root_, pred = 0, succ = 0;
        while (x) {
            int cmp = compare_query_node(q, qh, qi, x);
            if (cmp <= 0) {
                succ = x; x = tr[x].l;
            } else {
```

```
                pred = x; x = tr[x].r;
            }
        }
        return {pred, succ};
    }

public:
    DynamicSuffixArray(int max_capacity = 100000) {
        s.reserve(max_capacity + 10);
        tr.reserve(max_capacity + 10);
        hp.reserve(max_capacity + 10);
        pw.reserve(max_capacity + 10);
        s.push_back(0); tr.emplace_back();
        tr[0].sz = 0; tr[0].mn = INT_MAX; tr[0].mx = -1;
        hp.push_back(0); pw.push_back(1);
    }

    void push_front(char c) {
        int old_size = size();
        s.push_back(c); tr.emplace_back();
        int id = size(); tr[id].mn = tr[id].mx = id;
        root_ = insert(root_, id, getIndex(id - 1));
        ensure_powers(old_size + 1);
        hp.push_back(add(mul((u64)(unsigned char)c, pw[old_size]), hp[old_size]));
    }

    char pop_front() {
        if (size() <= 0) return 0;
        int id = size(), p = tr[id].p, tmp = merge(tr[id].l, tr[id].r);
        setP(tmp, p);
        if (p) {
            (tr[p].l == id ? tr[p].l : tr[p].r) = tmp;
            while (p) {
                upd(p);
                p = tr[p].p;
            }
        } else
            root_ = tmp;

        char c = s.back();
        tr.pop_back(); s.pop_back(); hp.pop_back();
        return c;
    }

    int count_occurrences(const string &q) { return get_matches(q).cnt; }
    int find_first(const string &q) { return get_matches(q).first_occ; }
    int find_last(const string &q) { return get_matches(q).last_occ; }

    int lcs(const string &q) {
        if (q.empty() || size() == 0) return 0;
        ensure_powers(max(size(), (int)q.size()));
        vector<u64> qh = make_hash(q);
        int best_len = 0, best_pos = 0;
        for (int i = 0; i < (int)q.size(); ++i) {
            auto [pred, succ] = pred_succ(q, qh, i);
            if (pred) {
```

```
                int len = lcp_query_node(q, qh, i, pred);
                if (len > best_len) { best_len = len; best_pos = i; }
            }
            if (succ) {
                int len = lcp_query_node(q, qh, i, succ);
                if (len > best_len) { best_len = len; best_pos = i; }
            }
        }
        // To restore the substring, you can use: q.substr(best_pos, best_len)
        return best_len;
    }
    int size() const { return (int)s.size() - 1; }
    char at(int i) const { return s[size() - i]; }
};
```

Generalized Suffix Automaton

```
struct generalized_sam {
    struct state : map<int, int> {
        int fail = -1, len{}, cnt = 0;
        bool ed = false;
    };
    vector<state> tr;
    int max_len = 0;

    explicit generalized_sam() : tr(1) {}

    int add(int c, int last) {
        if (tr[last].count(c)) {
            int q = tr[last][c];
            if (tr[last].len + 1 == tr[q].len) return q;
            int clone = int(tr.size());
            tr.emplace_back(tr[q]);
            tr[clone].cnt = 0, tr[clone].len = tr[last].len + 1;
            max_len = max(max_len, tr[clone].len);
            int p = last;
            while (~p && tr[p][c] == q) tr[p][c] = clone, p = tr[p].fail;
            tr[q].fail = clone;
            return clone;
        }

        int x = int(tr.size());
        tr.emplace_back();
        tr[x].len = tr[last].len + 1;
        max_len = max(max_len, tr[x].len);
        int p = last;
        while (~p && !tr[p].count(c)) tr[p][c] = x, p = tr[p].fail;
        if (p == -1) tr[x].fail = 0;
        else {
            int q = tr[p][c];
            if (tr[p].len + 1 == tr[q].len) tr[x].fail = q;
            else {
                int y = int(tr.size());
                tr.emplace_back(tr[q]);
                tr[y].cnt = 0, tr[y].len = tr[p].len + 1;
```

```
                max_len = max(max_len, tr[y].len);

                while (~p && tr[p][c] == q) tr[p][c] = y, p = tr[p].fail;
                tr[x].fail = tr[q].fail = y;
            }
        }
        return x;
    }

    void insert(const string& s) {
        int last = 0;
        for (auto i : s) {
            last = add(i, last);
            tr[last].cnt++;
        }
        tr[last].ed = true;
    }

    void init() {
        vector b(max_len + 1, vector(0, 0));
        for (int i = 0; i < tr.size(); ++i) b[tr[i].len].push_back(i);
        for (int l = max_len; l >= 1; --l) {
            for (int u : b[l]) {
                if (~tr[u].fail) {
                    tr[tr[u].fail].cnt += tr[u].cnt;
                    if (tr[u].ed) tr[tr[u].fail].ed = true;
                }
            }
        }
    }
};
```

Suffix Automaton

```
struct suffix_automaton {
    struct node {
        int len = 0, link = -1, fpos = 0, lpos = 0 ;
        map<char, int> nxt; // array<int,26> nxt;
        node(int len = 0, int link = -1, int fpos = 0)
        : len(len), link(link), fpos(fpos) { lpos = fpos ;}
    };

#define len(v) sa[v].len
#define link(v) sa[v].link
#define fpos(v) sa[v].fpos
#define lpos(v) sa[v].lpos
#define nxt(v) sa[v].nxt

    vector<node> sa;
    vector<vector<int>> g;
    int sz = 1, last = 0;

    suffix_automaton() {
        sa.emplace_back();
    }
    void extend(char c) {
```

```
        int curr = sz++;
        sa.emplace_back(len(last) + 1, -1, len(last) + 1);
        int p = last;
        while (p != -1 && !nxt(p).count(c)) {
            nxt(p)[c] = curr;
            p = link(p);
        }
        if (p == -1) {
            link(curr) = 0;
        } else {
            int q = nxt(p)[c];
            if (len(q) == len(p) + 1) {
                link(curr) = q;
            } else {
                int clone = sz++;
                sa.push_back(sa[q]);
                len(clone) = len(p) + 1;
                while (p != -1 && nxt(p).count(c) && nxt(p)[c] == q) {
                    nxt(p)[c] = clone;
                    p = link(p);
                }
                link(q) = link(curr) = clone;
            }
        }
        last = curr;
    }


    void prop() { // faster than build_tree and less memory
        int mxln = 0 ;
        for (auto &u : sa) mxln = max(mxln , u.len) ;
        vector<int> acc(mxln + 1) , order(sz) ;
        for (auto &u : sa) acc[u.len]++;
        for (int i = 1 ; i <= mxln ; i++) acc[i] += acc[i - 1] ;
        for (int i = sz - 1 ; i >= 0 ; i--) order[--acc[sa[i].len]] = i ;
        for (int i = sz - 1 ; i > 0 ; i--) {
            int node = order[i] , lnk = link(node) ;
            sa[lnk].lpos = max(sa[lnk].lpos , sa[node].lpos) ;
        }
    }
    int move (int v , char &c) { return (nxt(v).count(c) ? nxt(v)[c] : 0); }
    void move(int &v, int &len, char &c) {
        while (v > 0 && !nxt(v).count(c)) v = link(v) , len = len(v);
        if (nxt(v).count(c)) v = nxt(v)[c] , len++;
        else v = len = 0;
    }
};
string s = "ababa";
suffix_automaton sam;
for (char c : s) sam.extend(c);
sam.prop();
```

Tarjan SCC / Bridges / Art. Points

```
struct TarjanSCC {
```

```
    bool directed;
    int n, timer, sccCount;
    vector<int> in, low, sccId;
    vector<bool> inStack, isArticulation;
    stack<int> st;
    vector<pair<int, int>> bridges;
    vector<vector<int>> graph, sccs, dag;

    explicit TarjanSCC(int n, bool directed = false) :
            directed(directed), n(n), timer(0), sccCount(0) {
        graph.resize(n);
        in = low = sccId = vector(n, -1);
        inStack = isArticulation = vector(n, false);
    }

    void addEdge(int u, int v) {
        graph[u].push_back(v);
        if(!directed) graph[v].push_back(u);
    }

    void dfs(int u, int parent = -1) {
        in[u] = low[u] = ++timer;
        st.push(u), inStack[u] = true;

        int children = 0;
        for (int v : graph[u]) if(directed || v != parent) {
            if (in[v] == -1) {
                children++;
                dfs(v, u);

                if(low[v] > in[u]) bridges.emplace_back(u, v);
                if(parent != -1 && low[v] >= in[u]) isArticulation[u] = true;
                low[u] = min(low[u], low[v]);
            }
            else if(inStack[v])
                low[u] = min(low[u], in[v]);
        }
        if(parent == -1 && children > 1) isArticulation[u] = true;

        if (low[u] == in[u]) {
            sccs.emplace_back();
            int v;
            do {
                v = st.top(), st.pop(), inStack[v] = false;
                sccId[v] = sccCount;
                sccs.back().push_back(v);
            } while (v != u);
            ++sccCount;
        }
    }

    void init() { for(int i = 0; i < n; ++i) if(in[i] == -1) dfs(i); }

    void buildDAG() {
        dag.assign(sccCount, {});
        set<pair<int, int>> edgeSet;
```

```

        for (int u = 0, su, sv; u < n; ++u) {
            su = sccId[u];
            for(int v : graph[u]) {
                sv = sccId[v];
                if(su != sv && edgeSet.insert({su, sv}).second)
                    dag[su].push_back(sv);
            }
        }
    }
};
```

Static Range Mode Query (sqrt)

```
template <typename T>
struct StaticRangeModeQuery {
    int n, S, B;
    vector<T> val;
    vector<int> a, pos, pos_inv, st;
    vector<pair<int, int>> ans;

    StaticRangeModeQuery() = default;
    explicit StaticRangeModeQuery(const vector<T> &arr) {
        n = arr.size();
        S = max<int>(sqrt(n), 1);
        B = (n + S - 1) / S;

        val = arr;
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        int V = val.size();

        a.resize(n), st.assign(V + 1, 0);
        for (int i = 0; i < n; ++i)
            ++st[a[i] = lower_bound(val.begin(), val.end(), arr[i]) - val.begin()];

        for (int i = 0; i < V; ++i) st[i + 1] += st[i];

        pos.resize(n), pos_inv.resize(n);
        for (int i = n; i--;)
            pos[pos_inv[i] = --st[a[i]]] = i;

        ans.assign((B + 1) * (B + 1), {0, 0});
        vector<int> cnt(V);
        for (int l = 0; l <= B; ++l) {
            cnt.assign(V, 0);
            pair<int, int> cur{0, 0};
            for (int r = l + 1; r <= B; ++r) {
                for (int i = (r - 1) * S, end = min(n, i + S); i < end; ++i) {
                    pair<int, int> cand{++cnt[a[i]], a[i]};
                    if (cand > cur) cur = cand;
                }
                ans[l * (B + 1) + r] = cur;
            }
        }
```

```
    }

    pair<T, int> query(int l, int r) const {
        int lb = (l + S - 1) / S, rb = r / S;
        auto [freq, res] = ans[lb * (B + 1) + rb];

        for (int i = l, end = min(r, lb * S); i < end; ++i) {
            int v = a[i], idx = pos_inv[i];
            while (idx + freq < st[v + 1] && pos[idx + freq] < r) ++freq, res = v;
        }
        for (int i = r, end = max(l, rb * S); i-- > end;) {
            int v = a[i], idx = pos_inv[i];
            while (idx - freq >= st[v] && pos[idx - freq] >= l) ++freq, res = v;
        }
        return {val[res], freq};
    }

    pair<T, int> operator()(int l, int r) const { return query(l, r); }
};
```

Lyndon Factorization

```
// returns an array of end points of each word (exclusive)
// s = [0,v[0]) + [v[0],v[1]) + ...
vector<int> duval(vector<int> &s){
    int n = sz(s);
    vector<int> v;
    for(int i = 0; i < n; ){
        int j = i + 1, k = i;
        while(j < n && s[k] <= s[j]){
            if(s[k] == s[j]) j++, k++;
            else if(s[k] < s[j]) j++, k = i;
        }
        while(i <= k){
            i += j - k;
            v.push_back(i);
        }
    }
    return v;
}
```

kinetic_tournament

```
// Suppose that you have an array containing pairs of nonnegative integers,
// A[i] and B[i]. You also have a global parameter T, corresponding to the
// "temperature" of the data structure. Your goal is to support the following
// queries on this data:
//
//   - update(i, a, b): set A[i] = a and B[i] = b
//   - query(s, e): return min{s <= i <= e} A[i] * T + B[i]
//   - heaten(new_temp): set T = new_temp
//       [precondition: new_temp >= current value of T]
//
// (For simplicity, we set A[i] = 0 and B[i] = LLONG_MAX for uninitialized
```

```
// entries, which should not change the query results.)
//
// This allows you to essentially do arbitrary lower convex hull queries on a
// collection of lines, as well as any contiguous subcollection of those lines.
// This is more powerful than standard convex hull tricks and related data
// structures (Li-Chao Segment Tree) for three reasons:
//
//   - You can arbitrarily remove/edit lines, not just add them.
//   - Dynamic access to any subinterval of lines, which lets you avoid costly
//     merge small-to-large operations in some cases.
//   - Easy to reason about and implement from scratch, unlike dynamic CHT.
//
// The tradeoff is that you can only query sequential values (temperature is
// only allowed to increase) for amortization reasons, but this happens to be
// a fairly common case in many problems.
//
// Time complexity:
//
//   - query: O(log n)
//   - update: O(log n)
//   - heaten: O(log^2 n)  [amortized]
//

#include <bits/stdc++.h>
using namespace std;

template <typename T = int64_t>
class kinetic_tournament {
    const T INF = numeric_limits<T>::max();
    typedef pair<T, T> line;

    size_t n;         // size of the underlying array
    T temp;           // current temperature
    vector<line> st;  // tournament tree
    vector<T> melt;   // melting temperature of each subtree

    inline T eval(const line& ln, T t) {
        return ln.first * t + ln.second;
    }

    inline bool cmp(const line& line1, const line& line2) {
        auto x = eval(line1, temp);
        auto y = eval(line2, temp);
        if (x != y) return x < y;
        return line1.first < line2.first;
    }

    T next_isect(const line& line1, const line& line2) {
        if (line1.first > line2.first) {
            T delta = eval(line2, temp) - eval(line1, temp);
            T delta_slope = line1.first - line2.first;
            assert(delta > 0);
            T mint = temp + (delta - 1) / delta_slope + 1;
            return mint > temp ? mint : INF;  // prevent overflow
        }
        return INF;
    }
```

```
69
70      void recompute(size_t lo, size_t hi, size_t node) {
71          if (lo == hi || melt[node] > temp) return;
72
73          size_t mid = (lo + hi) / 2;
74          recompute(lo, mid, 2 * node + 1);
75          recompute(mid + 1, hi, 2 * node + 2);
76
77          auto line1 = st[2 * node + 1];
78          auto line2 = st[2 * node + 2];
79          if (!cmp(line1, line2))
80              swap(line1, line2);
81          st[node] = line1;
82
83          melt[node] = min(melt[2 * node + 1], melt[2 * node + 2]);
84          if (line1 != line2) {
85              T t = next_isect(line1, line2);
86              assert(t > temp);
87              melt[node] = min(melt[node], t);
88          }
89      }
90
91      void update(size_t i, T a, T b, size_t lo, size_t hi, size_t node) {
92          if (i < lo || i > hi) return;
93          if (lo == hi) {
94              st[node] = {a, b};
95              return;
96          }
97          size_t mid = (lo + hi) / 2;
98          update(i, a, b, lo, mid, 2 * node + 1);
99          update(i, a, b, mid + 1, hi, 2 * node + 2);
100         melt[node] = 0;
101         recompute(lo, hi, node);
102     }
103
104     T query(size_t s, size_t e, size_t lo, size_t hi, size_t node) {
105         if (hi < s || lo > e) return INF;
106         if (s <= lo && hi <= e) return eval(st[node], temp);
107         size_t mid = (lo + hi) / 2;
108         return min(query(s, e, lo, mid, 2 * node + 1),
109             query(s, e, mid + 1, hi, 2 * node + 2));
110     }
111
112 public:
113     // Constructor for a kinetic tournament, takes in the size n of the
114     // underlying arrays a[..], b[..] as input.
115     kinetic_tournament(size_t size) : n(size), temp(0) {
116         assert(size > 0);
117         size_t seg_size = ((size_t) 2) << (64 - __builtin_clzll(n - 1));
118         st.resize(seg_size, {0, INF});
119         melt.resize(seg_size, INF);
120     }
121
122     // Sets A[i] = a, B[i] = b.
123     void update(size_t i, T a, T b) {
124         update(i, a, b, 0, n - 1, 0);
```

```
    }

    // Returns min{s <= i <= e} A[i] * T + B[i].
    T query(size_t s, size_t e) {
        return query(s, e, 0, n - 1, 0);
    }

    // Increases the internal temperature to new_temp.
    void heaten(T new_temp) {
        assert(new_temp >= temp);
        temp = new_temp;
        recompute(0, n - 1, 0);
    }
};
```

Discrete Root [x ^ k = a (mod m)]

```
// returns any or all numbers x such that x ^ k = a (mod m)
// existence: a = 0 is trivial, and if a > 0: a ^ (phi(m) / gcd(k, phi(m))) == 1 mod m
// if solution exists, then number of solutions = gcd(k, phi(m)).
// here m is prime, but it will work for any integer which has a primitive root
int discrete_root(int k, int a, int m) {
  if (a == 0) return 1;
  int g = primitive_root(m);
  int phi = m - 1; // m is prime
  // run baby step-giant step
  int sq = (int) sqrt(m + .0) + 1;
  vector<pair<int, int>> dec(sq);
  for (int i = 1; i <= sq; ++i) dec[i - 1] =
    make_pair(power(g, 1LL * i * sq % phi * k % phi, m), i);
  sort(dec.begin(), dec.end());
  int any_ans = -1;
  for (int i = 0; i < sq; ++i) {
    int my = power(g, 1LL * i * k % phi, m) * 1LL * a % m;
    auto it = lower_bound(dec.begin(), dec.end(), make_pair(my, 0));
    if (it != dec.end() && it->first == my) {
      any_ans = it->second * sq - i;
      break;
    }
  }
  if (any_ans == -1) return -1; //no solution
  // for any answer
  int delta = (m - 1) / __gcd(k, m - 1);
  return power(g, any_ans % delta, m);
  // // for all possible answers
  // int delta = (m - 1) / __gcd(k, m - 1);
  // vector<int> ans;
  // for (int cur = any_ans % delta; cur < m-1; cur += delta)
    // ans.push_back(power(g, cur, m));
  // sort(ans.begin(), ans.end());
  // // assert(ans.size() == __gcd(k, m - 1))
  // return ans;
}
```

Range Progression Sum

```
struct tag {
    ll a = 0, d = 0;
    void apply(const tag &p) {
        a += p.a; d += p.d;
    }
};
struct info {
    ll sum = 0;
    info(ll sum = 0) : sum(sum) {}
    void apply(const tag &lz, int lx, int rx) {
        ll len = rx - lx + 1;
        ll first = lz.a + lz.d * lx;
        ll last = lz.a + lz.d * rx;
        sum += (first + last) * len / 2;
    }
    friend info operator+(const info &l, const info &r) {
        return {l.sum + r.sum};
    }
};

// add A, A+D, A+2D, ... to [l,r]
st.applyRange(l, r, {
    A - 1LL * D * l,
    D
});
```

Count a prime in n!

```
ll count_p_in_nfact(ll p, ll n) {
    ll res = 0, q = p;
    while (q <= n) res += n / q, q *= p;
    return res;
}
```

Power Tower (Tetration)

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
struct chash {
    const int RANDOM = (ll)(make_unique<char>().get()) ^
        chrono::high_resolution_clock::now().time_since_epoch().count();

    static unsigned long long hash_f(unsigned long long x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    static unsigned hash_combine(unsigned a, unsigned b) {
        return a * 31 + b;
    }
    int operator()(int x) const {
```

```
        return hash_f(x)^RANDOM;
    }
};
gp_hash_table<ll, ll, chash> phi_cache;
ll get_phi(ll n) {
    if (phi_cache.find(n) != phi_cache.end()) return phi_cache[n];
    ll ans = n, m = n;
    for (ll i = 2; i * i <= m; i++) {
        if (m % i == 0) {
            while (m % i == 0) m /= i;
            ans -= ans / i; // Condensed standard phi formula
        }
    }
    if (m > 1) ans -= ans / m;
    return phi_cache[n] = ans;
}
ll ext_mod(__int128_t x, ll m) {
    return x < m ? (ll)x : (ll)(x % m) + m;
}
ll ext_pow(ll base, ll exp, ll mod) {
    base = ext_mod(base, mod);
    ll res = ext_mod(1, mod);
    while (exp > 0) {
        if (exp & 1) res = ext_mod((__int128_t)res * base, mod);
        base = ext_mod((__int128_t)base * base, mod);
        exp >>= 1;
    }
    return res;
}
ll solve_tower(const vector<ll> &a, int l, int r, ll m) {
    if (m == 1) return 1;
    if (l == r) return ext_mod(a[l], m);
    return ext_pow(a[l], solve_tower(a, l + 1, r, get_phi(m)), m);
}
// a[l] ^ (a[l+1] ^ (a[l+2] ... ^ (a[r]) ) )
ll power_tower(const vector<ll> &a, int l, int r, ll m) {
    return solve_tower(a, l, r, m) % m;
}
```

Stress

```
#include "bits/stdc++.h"

using namespace std;
#define ll long long
#define int long long

random_device rd;
mt19937_64 mt(rd());

ll rnd(ll l, ll r) { return uniform_int_distribution<ll>(l, r)(mt); }

const int LIMIT = 1e6;

void generate() {
```

```
  ofstream cout("test.txt");

  int n = rnd(1, 10);
  cout << n << '\n';
  for (int i = 0; i < n; ++i) {
    cout << rnd(1, 1e5) << '␣';
  }

  cout.close();
}

int32_t main() {
  system("g++␣-lm␣-O3␣-std=c++17␣-DLOCAL␣-pipe␣-o␣main␣../main.cpp");
  system("g++␣-lm␣-O3␣-std=c++17␣-pipe␣-o␣brute␣../brute.cpp");
//  system("g++ -lm -O3 -std=c++17 -pipe -o gen ../gen.cpp"); // run gen file ?

  for (int tc = 1; tc <= LIMIT; ++tc) {
    cerr << "Case␣" << tc << '\n';

//    system("gen >test.txt"); /// file
//    generate(); /// function

    system("main␣<test.txt␣>wa.txt");
    if (system("brute␣<test.txt␣>ac.txt"))break;
    ifstream acs("ac.txt");
    ifstream was("wa.txt");
    string ac, wa;
    getline(was, wa, (char) EOF);
    getline(acs, ac, (char) EOF);
    was.close();
    acs.close();
    if (ac != wa) break;
  }
}
```